Hello, world!

A Programming Language

Two Variables

- `x`, `y`

Three Operations

- `x++`
- `x--`
- `x = 0 ? L1 : L2`

Example Program

(What *does* it do?)

```
L1: x++
    y--
    y = 0 ? L2 : L1
L2: ...
```

The above language is "equivalent to" every PL!

But good luck writing

- … QuickSort
- … Fortnite
- … Spotify!

So Why Study Programming Languages?

Edsger Dijkstra "On the foolishness of natural language programming", 1978

> *From one gut feeling I derive much consolation: I suspect that machines to be programmed in our native tongues — be it Dutch, English, American, French, German, or Swahili— are as damned difficult to make as they would be to use.*

Fifty years later …

the elephant in the room

the elephant in the room

… Won't LLMs like `ChatGPT`, `Claude`, `Grok`, … let us **vibe code** all the programs?

Greg Brockman, President, CTO & Co-Founder, OpenAI

Not just OpenAI…

… similar stories from Microsoft, Meta, Amazon, etc.

How to ensure "if it compiles, it is ~correct"?

> *The virtue of formal texts is that their manipulations, in order to be legitimate,* ***need to satisfy only a few simple rules;*** *they are, when you come to think of it, an amazingly effective tool for ruling out all sorts of nonsense that, when we use our native tongues, are almost impossible to avoid.*

Goal: Programming Language as a means of Abstraction

How to specify

- What a program **does**?
- What a program **does not**?

Course Goals

130 Brain

130 Brain

New languages come (and go …)

There was no

- Java 30 years ago
- C# 25 years ago
- Rust 15 years ago
- WebAssembly 5 years ago

Goal: Learn the Anatomy of PL

Anatomy

Anatomy

- What makes a programming language?
- Which features are **fundamental** and which are **syntactic sugar**?

Goal: Learn New Languages / Constructs

Musical Score

Musical Score

New ways to **describe** and **organize** computation, to create programs that are:

- **Correct**
- **Readable**
- **Extendable**
- **Reusable**

Goal: How to Design new Languages

New languages being designed in industry as we speak:

- Flow, React @ Facebook
- Rust @ Mozilla
- TypeScript @ Microsoft
- Swift @ Apple
- WebAssembly @ Google + Mozilla + Microsoft
- Verse @ Epic Games

Buried in every large system is a (domain-specific) language

- DB: SQL
- Word, Excel: Formulas, Macros, VBScript
- Emacs: LISP
- Latex, shell scripts, makefiles, …

If you work on a large system, you **will** design a new PL!

Goal: How to Design new Languages

New languages like `Verse` @ Epic Games

Goal: Enable You To Choose Right PL

But isn't that decided by

- Libraries
- Standards
- Hiring
- Your Boss?!

Yes.

My goal: Educate tomorrow's leaders so you'll make **informed** choices.

The Crew

Profs

Ranjit Jhala

- Professor at CSE since 2005
- Research: Tools and Techniques to make programs better

The Crew

Teaching Assistants

- Cole Kurashige

Tutors

- Yuehua Xie
- Alain Zhang

Course Syllabus

Functional Programming

- Lambda calculus (2 weeks)
- Functional Programming (4 weeks)
- Building an Interpreter (3 weeks)

Logic Programming

- Prolog (~1 week, maybe, but doubtful!)

QuickSort in C

```
void sort(int arr[], int beg, int end){
  if (end > beg + 1){
    int piv = arr[beg];
    int l = beg + 1;
    int r = end;
    while (l != r-1)
      if(arr[l] <= piv) l++;
      else swap(&arr[l], &arr[r--]);
    if(arr[l]<=piv && arr[r]<=piv)
      l=r+1;
    else if(arr[l]<=piv && arr[r]>piv)
      {l++; r--;}
    else if (arr[l]>piv && arr[r]<=piv)
      swap(&arr[l++], &arr[r--]);
    else r=l-1;
    swap(&arr[r--], &arr[beg]);
    sort(arr, beg, r);
    sort(arr, l, end);
  }
}
```

QuickSort in Haskell

```
sort []     = []
sort (x:xs) = sort ls ++ [x] ++ sort rs
  where
    ls      = [ l | l <- xs, l <= x ]
    rs      = [ r | r <- xs, x <  r ]
```

(**Note:** not a wholly fair comparison…)

Course Logistics

webpage

- Calendar
- Lecture notes
- Programming assignments

piazza

- Go-to place if you have a question or need help

Policies

- No podcasting.
- No screens (phones, laptops) in lecture.
- Yes attendance & class participation (worksheets).
- Yes exams must be done at allotted time and location.

Grading

- 10% Class participation (*worksheets*)
- 30% Programming Assignments (*codespaces*)
- 30% Two Midterm **Tu 1/27** and **Thu 2/19** (*in lecture*)
- 30% Final **Tu 3/18**
- 05% Piazza Extra Credit (*top 5* best participants)

Class participation (10%)

- “In class” worksheets handed out each lecture
- Handed in at the end of the lecture
- Turn in 75% of the worksheets to get full credit
- Responses will be graded on participation (not correctness)

Assignments (30%)

6 programming assignments

- Released online
- At least a week before due date
- Via github classroom + codespaces

Six late days

- used as **whole unit**
- 5 mins late = 1 late day
- at most 4 per assignment

No Groups

- Assignments must be submitted individually via github
- Ok to discuss with classmates, but solution must be your own

Exams (15 + 15 + 30%)

- Must be done at allotted time and location
- 2-sided ``cheat sheet”
- The final is cumulative

Your Resources

Discussion section

- Th 6:00-6:50pm (CENTER 212)

Office hours

- Every day, check **CANVAS calendar**

Piazza

- We answer during work hours and office hours (M-F)

No text

- Online lecture notes and links

Academic Integrity

Programming assignments: do not copy from classmates or from previous years

Exams done **alone**

- Zero Tolerance
- Offenders punished ruthlessly
- Please see academic integrity statement

Students with Disabilites

Students requesting accommodations for this course due to a disability or current functional limitation must provide a current **Authorization for Accommodation (AFA)** letter issued by the Office for Students with Disabilities (OSD) which is located in University Center 202 behind Center Hall.

Students are required to present their AFA letters to Faculty (please make arrangements to contact me privately) and to the CSE OSD Liaison Christina Rontell in advance so that accommodations may be arranged.

Generated by Hakyll, template by Armin Ronacher, suggest improvements here.